

## Can you use `*` to print everything?

In Python, the `*` (splat) operator **unpacks** a collection into separate arguments. When used inside `print()`, it's like taking every item out of its box and handing it to the print function one by one.

### 1. The "Yes" Category (Works Great)

Anything that is a **sequence** or **iterable** works perfectly.

- **Lists:** `print(*[1, 2, 3])` → `1 2 3`
- **Tuples:** `print(*(1, 2, 3))` → `1 2 3`
- **Strings:** `print(*"HEY0")` → `H E Y 0` (It treats the string as a list of characters)
- **Sets:** `print(*{1, 2, 3})` → `1 2 3` (But remember: order is random!)
- **Ranges:** `print(*range(5))` → `0 1 2 3 4`

### 2. The "Tricky" Category (Dictionaries)

If you try to "splat" a dictionary, it only looks at the **Keys**, not the values.

```
user = {"name": "Bro", "age": 25}
print(*user)
# Output: name age
```

If you want the values, you have to be specific: `print(*user.values())`.

### 3. The "No" Category (Fails)

You cannot use `*` on items that aren't "containers." If it doesn't have multiple parts inside, you can't unpack it.

- **Integers:** `print(*10)` → `TypeError: 'int' object is not iterable`
- **Floats:** `print(*10.5)` → `TypeError: 'float' object is not iterable`
- **Booleans:** `print(*True)` → `TypeError: 'bool' object is not iterable`

Why use `*` instead of just printing the variable?

| Method   | Syntax                         | Output                                             |
|----------|--------------------------------|----------------------------------------------------|
| Standard | <code>print([1, 2, 3])</code>  | <code>[1, 2, 3]</code> (Shows brackets and commas) |
| Unpacked | <code>print(*[1, 2, 3])</code> | <code>1 2 3</code> (Clean, space-separated text)   |

**Pro-Tip: Changing the Separator**

When you use `*`, you can also use the `sep` argument to decide what goes between the items:

```
words = ["I", "love", "Python"]  
print(*words, sep="-")  
# Output: I-love-Python
```